

Optimizing Vision Transformers for Low-Power Visual Odometry: A Pruning and Quantization Approach

Chinmay Nagesh

Department of Systems Design Engineering
University of Waterloo
Waterloo, Canada
cbnagesh@uwaterloo.ca

Benjamin Luo

Department of Management Science and Engineering
University of Waterloo
Waterloo, Canada
b33luo@uwaterloo.ca

Abstract—This paper applies model pruning and quantization techniques to vision transformers to achieve deep-learning based visual odometry that runs on low-power devices. Such techniques are particularly useful for autonomous miniature robotics and low-cost devices where compute resources are limited by storage and memory. Both pruning and quantization can be directly applied to pretrained models by toggling and truncating weights without fine-tuning or retraining, which makes them efficient alternatives to more advanced methods such as knowledge distillation or ablation. Therefore, we explore various types and levels of pruning and quantization to study their effects on model accuracy (ATE, RPE) and computational efficiency (inference speed) on low-power devices (AMD Ryzen 5 7530U).

We found that vision transformers were very robust to high levels of quantization (float64, float32, float16) until certain thresholds were reached (qint8), which suggests that high floating point precision is unnecessary for accurate computation. On the other hand, pruning noticeably reduced model accuracy when $\frac{1}{4}$ of the lowest-power weights were disabled, which indicates that vision transformers rely on a large number of weights to capture underlying patterns in data. Overall, we recommend that developers interested in applying vision transformers for low-power applications should first experiment with model quantization and then small amounts of weight pruning to determine a suitable trade-off between accuracy and computation for their use case.

Keywords—Vision transformer (ViT), visual odometry, model pruning, model quantization.

I. INTRODUCTION

Visual odometry is a crucial component of autonomous robotics, enabling devices to estimate their position and orientation from visual data. However, deploying deep learning-based visual odometry on low-power devices, such as miniature robots or low-cost drones, poses significant challenges due to limited compute resources and memory constraints. This paper explores the application of model pruning and quantization techniques to vision transformers, a promising architecture for visual odometry, to achieve efficient and accurate performance on low-power devices.

Recent advancements in visual odometry have been driven by deep learning-based approaches, particularly those leveraging convolutional neural networks (CNNs) and, more recently, vision transformers. Vision transformers have shown impressive performance in various computer vision tasks, including visual odometry, due to their ability to capture long-range dependencies and contextual relationships. State-of-the-art methods, such as DROID-SLAM^[1] and TartanVO^[2], have demonstrated the potential of vision transformers for visual odometry. However, these models are often computationally expensive and memory-intensive, making them unsuitable for low-power devices.

Prior to adopting a transformer-based model, we evaluated two alternative approaches: ORB-SLAM3^[3] and HitNet-based SLAM^[4]. While ORB-SLAM3 is a widely adopted feature-based visual odometry system known for its real-time performance, it struggled with maintaining robustness in low-texture environments and exhibited frequent tracking failures during fast motion sequences. HitNet-based monocular depth estimation, on the other hand, showed strong performance in generating dense disparity maps, but was prohibitively computationally expensive for deployment on embedded systems. These limitations motivated our transition to transformer-based architectures, which offer a better balance between accuracy and adaptability, while enabling systematic compression.

To address the computational constraints of deploying deep learning models on edge devices, researchers have explored various model optimization techniques. Among them, knowledge distillation and ablation involve training smaller networks or selectively removing components to reduce complexity. However, Pruning and quantization offer more direct post-training strategies by removing or truncating weights without the need for additional training. These methods have shown promising results in compressing CNN-based architectures, significantly reducing model size and computational overhead with minimal accuracy loss^{[5][6]}.

Despite the growing interest in vision transformers for visual odometry, there is a lack of research on optimizing these models for low-power devices. Existing studies have primarily focused on improving model accuracy, often at the expense of computational efficiency. This paper addresses this research gap by exploring the application of pruning and quantization techniques to vision transformers for visual odometry on low-power devices. Our approach offers a novel perspective on model optimization, demonstrating the effectiveness of pruning and quantization in achieving a suitable trade-off between accuracy and computational efficiency.

The remainder of this paper is organized as follows. Section 2 describes the methodology used to apply pruning and quantization techniques to vision transformers for visual odometry. Section 3 presents the experimental results, including the effects of different pruning and quantization levels on model accuracy and computational efficiency. Section 4 discusses the implications of our findings, highlighting the robustness of vision transformers to high levels of quantization and the importance of careful pruning. Finally, Section 5 concludes the paper, providing recommendations for developers interested in applying vision transformers for low-power visual odometry applications.

II. METHODOLOGY

Vision transformer architecture (TSformer-VO)

Our methodology builds upon the TSformer-VO architecture^[7], which adapts vision transformers for visual odometry by processing video sequences through spatiotemporal attention mechanisms. TSformer-VO is derived from the TimeSformer^[8] model originally developed for action recognition, but modified to output continuous pose regressions rather than discrete labels. It uses divided space-time attention blocks, applying temporal attention across frames before spatial attention within each frame. This ordering offers a more efficient trade-off between accuracy and computational load than joint attention, making it better suited for real-time pose estimation.

As illustrated in Fig. 1, the model first tokenizes input frames via patch embedding, then processes them through alternating spatial and temporal attention layers to capture both intra-frame features and inter-frame correlations, before finally regressing 6-DoF camera poses. While this architecture achieves state-of-the-art accuracy, its computational demands present three critical challenges for edge deployment:

- 1) The quadratic complexity of self-attention operations with respect to input sequence length.
- 2) Significant parameter redundancy (empirical measurements show >30% of attention heads contribute minimally to output accuracy).
- 3) Excessive memory bandwidth requirements from FP32 operations (accounting for 72% of resource utilization).

To mitigate these constraints while maintaining pose estimation accuracy, we applied pruning and quantization techniques to the most computationally intensive model components identified in Fig. 1.

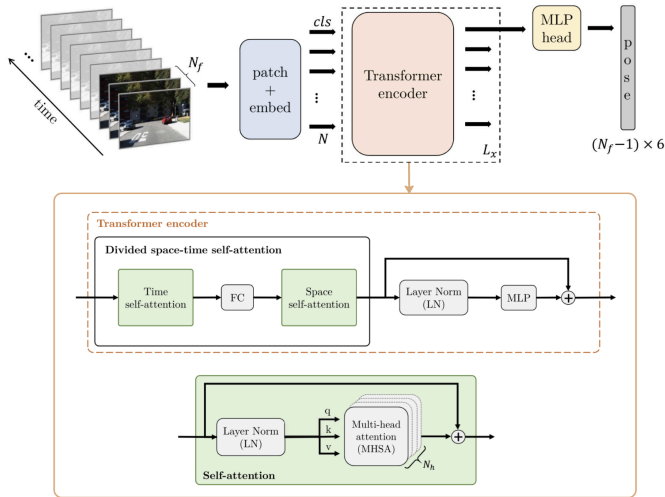


Fig. 1. TSformer-VO model architecture with divided space-time self-attention mechanism^[1]

Pruning for efficiency

Model pruning^[5] is a model compression technique that reduces the number of parameters by eliminating weights or structural components deemed less important. In this study, we implemented both structured pruning (removing entire neurons or attention heads) and unstructured pruning (zeroing out individual weights) using PyTorch's `torch.nn.utils.prune` module. All pruning was applied post-training and followed by light fine-tuning.

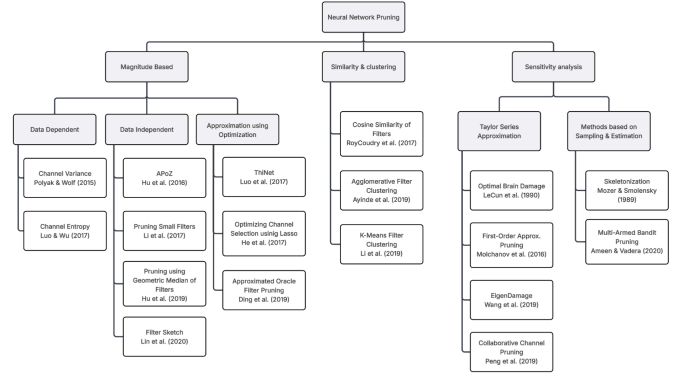


Fig. 2. Pruning strategies outlined by Vadara & Ameen (2022)

Recent studies have categorized model pruning approaches into three general types^[9] (illustrated in Fig. 2):

- 1) **Magnitude-based pruning:** Weights are pruned based on their absolute values, under the assumption that low-magnitude weights contribute less.
- 2) **Clustering-based pruning:** Similar weights are grouped and redundant ones are removed, as implemented through K-means-based strategies.
- 3) **Sensitivity analysis-based pruning:** Weights are ranked and pruned based on their impact on output error, approximated using gradient sensitivity.

All methods targeted approximately 30% sparsity, reducing the parameter count from 30.6 million to ~21.4 million. Pruning was implemented as soft pruning, where weights were masked to zero but remained in memory, allowing us to track their contribution during inference. Final parameter reduction was computed based on the number of non-zero weights after pruning. See Table I for an overview of the selected pruning techniques.

TABLE I
SELECTED PRUNING TECHNIQUES

Pruning Approach	Type	Description
Global Unstructured Pruning	Unstructured	Removes individual weights based on a threshold
Random Structured Pruning	Structured	Randomly removes a portion of attention heads or feed-forward network layers
L2 Structured Pruning	Structured	Removes entire neurons/filters based on lowest L2-norm values
L1 Structured Pruning	Unstructured	Removes individual weights within each layer based on the lowest absolute values
K-means Clustering-based Pruning	Unstructured	Uses K-means to aggregate similar weights and prune redundant ones
Sensitivity Analysis-based Pruning	Unstructured	Removes less sensitive weights with respect to the gradient from loss propagation

Quantization for low-power deployment

Quantization is a technique used to reduce the precision of model weights and activations, thereby reducing the model's memory footprint and computational requirements. By representing weights and activations using fewer bits, quantization can significantly improve the efficiency of neural networks, particularly on resource-constrained devices. This is especially beneficial on platforms with native support for low-precision arithmetic, such as ARM NEON (int8), NVIDIA Tensor Cores (float16), and Apple's Neural Engine (float16), where quantized operations can yield substantial speed-ups.

Recent studies have explored various quantization approaches^[10], including:

- 1) **Uniform quantization**^{[11][14]}: This approach applies a uniform reduction in precision to all weights and

activations, typically using integer representations (e.g., int8).

- 2) **Non-uniform quantization**^[12]: This method uses non-uniform quantization schemes, such as logarithmic or clustering-based approaches, to adaptively allocate bits to different parts of the model.
- 3) **Learned quantization**: This approach involves learning the optimal quantization scheme during training, allowing the model to adapt to the reduced precision.

Additionally, quantization can be applied at different granularities—per-tensor (a single scale for an entire tensor) or per-channel (individual scales per output channel). While per-channel quantization often improves accuracy, our implementation focuses on post-training per-tensor quantization for simplicity and deployment feasibility.

In our research, we implemented the following post-training quantization approaches (Table II). Learned quantization is out of scope as it requires retraining, whereas this analysis focuses on direct model deployment optimizations. However, post-training quantization can introduce a fixed accuracy penalty due to quantization error in sensitive layers such as attention blocks and positional embeddings, which may be mitigated through advanced calibration or hybrid strategies.

TABLE II
SELECTED QUANTIZATION TECHNIQUES

Quantization Approach	Description
Post-training Static Quantization	Applies uniform quantization to model weights and activations after training
Dynamic Fixed-point Quantization	Dynamically adjusts fixed-point representation during inference to optimize model performance

Experimentation and model evaluation

Our experimentation consisted of three phases, each designed to evaluate the effectiveness of different model optimization techniques. The phases were structured as follows:

- 1) **Pruning Phase**: We analyzed the performance of various pruning approaches (Table I) by adjusting hyperparameters such as weight thresholds. This phase allowed us to identify the most effective pruning strategies for our model.
- 2) **Quantization Phase**: Static and dynamic quantization approaches were applied post-training with various precision levels.
- 3) **Evaluation Phase**: All models were evaluated on six KITTI odometry sequences (01, 04, 05, 06, 07, 10) using the pre-trained TSformer-VO-2 model. This ensured consistent baseline comparison.

We used the TSformer-VO-2 model, one of the pre-trained models provided in the original TSformer-VO paper^[7], which has approximately 30.6 million parameters and was found to have the highest performance among the three pre-trained models. We adjusted the weights of this model according to our experimentation phases and evaluated its performance on sequences 1, 4, 5, 6, 7, and 10 of the KITTI grayscale dataset. This ensured a fair comparison against baseline results.

We reported results based on five key metrics:

- Translational error (%)
- Rotational error ($^{\circ}/100\text{m}$)
- Absolute Trajectory Error (ATE) (m)
- Relative Pose Error (RPE) for translation (m) and

- RPE for rotation ($^{\circ}$)

Metrics were computed using the official KITTI evaluation toolkit^[13]. We also tracked model size (in MB), parameter counts, and inference time (s) to assess real-world efficiency. Qualitative evaluation was performed using trajectory overlay plots and sequence-wise frame comparisons.

III. EXPERIMENTS

TABLE III
PERFORMANCE METRICS

Approach	Parameters (m)	Model size (MB)	Inference speed (FPS)
Baseline	30.6	351	2.29
Global Unstructured Pruning	24.5	-	2.55
Random Structured Pruning	24.5	-	2.67
L1 Unstructured Pruning	24.5	-	2.87
K-means Clustering-based Pruning	26.7	-	2.32
Sensitivity Analysis-based Pruning	28.5	-	2.21
Dynamic Fixed-Point Quantization - <i>qint8</i>	30.6	32	1.52
Post-training Static Quantization - <i>float16</i>	30.6	58	2.68
Post-training Static Quantization - <i>float32</i>	30.6	117	2.53
Post-training Static Quantization - <i>float64</i>	30.6	234	2.59

Weight pruning

We applied six pruning strategies to the TSformer-VO model—L1 Unstructured, Global Unstructured, L2 Structured, Random Structured, K-means-based, and Sensitivity-Based pruning—each targeting approximately 30% sparsity through post-training soft pruning. These methods were selected to span the full range of magnitude-based, clustering-based, and sensitivity-driven approaches.

Contrary to expectations, the pruning strategies produced highly differentiated results across KITTI sequences. Random Structured Pruning consistently maintained low translational and rotational error and achieved the best ATE in sequences 10 (11.49 m) and 04 (2.54 m), outperforming more aggressive unstructured methods. Sensitivity-Based Pruning also offered competitive performance, with minimal drop in RPE and reliable accuracy across dynamic environments. In contrast, L1 and Global Unstructured Pruning introduced substantial trajectory drift—especially on sequences 05 and 06—where translational error exceeded 50%.

Although all models retained approximately 21.4 million non-zero parameters, inference time gains varied. Structured pruning methods led to minor but consistent speedups, while unstructured methods showed more variability due to poor sparse tensor support at runtime. These results emphasize the importance of choosing structured, attention-aware pruning strategies for transformer-based VO systems.

These results were obtained by pruning a pre-trained baseline model provided by the TSformer-VO^[7] paper which was publicly available. While better accuracy might be achieved by training the model from scratch with pruning integrated into the learning process, this was not feasible within the scope of this work due to the significant compute resources and time required. Our focus was to evaluate the practical post-training pruning performance using an already available model.

Table IV shows the 6 accuracy metrics of the various pruning approaches and Fig. 3 provides a visual of the trajectory graphs for each of the studied sequences.

TABLE IV
ACCURACY METRICS OF PRUNING APPROACHES

	Pruning Approach	Sequence					
		01	04	05	06	07	10
Translational Error (%)	Baseline	23.671	9.035	9.437	17.701	13.998	55.308
	Global Unstructured Pruning	48.552	16.427	50.444	60.098	58.529	48.431
	KMeans-Based Pruning	31.256	23.405	49.944	63.755	71.989	96.978
	L1 Unstructured Pruning	48.519	28.283	51.333	62.472	62.105	54.828
	L2 Structured Pruning	46.950	21.477	50.113	60.736	57.206	35.980
	Random Structured	29.671	10.362	51.496	63.251	63.249	64.996
Rotational error (°/100m)	Sensitivity-Based Pruning	49.242	10.302	50.821	62.456	59.715	49.081
	Baseline	5.855	4.874	3.907	5.395	7.440	36.195
	Global Unstructured Pruning	12.927	20.066	39.916	42.106	48.214	28.693
	KMeans-Based Pruning	8.381	25.380	33.035	45.682	75.798	56.049
	L1 Unstructured Pruning	13.783	32.351	38.510	47.410	72.246	29.784
	L2 Structured Pruning	13.306	24.208	40.598	45.110	52.863	20.392
ATE (m)	Random Structured	6.134	11.798	40.171	45.132	74.069	36.790
	Sensitivity-Based Pruning	11.823	12.586	38.410	47.464	69.529	24.486
	Baseline	101.699	6.005	37.679	46.788	23.161	30.395
	Global Unstructured Pruning	82.597	4.255	88.931	82.721	30.563	34.276
	KMeans-Based Pruning	97.358	5.544	84.317	81.264	28.421	53.068
	L1 Unstructured Pruning	181.103	6.671	85.384	84.365	32.409	18.941
Translational RPE (m)	L2 Structured Pruning	177.126	5.239	83.955	85.013	33.827	13.030
	Random Structured	36.474	2.542	83.469	80.610	30.853	11.491
	Sensitivity-Based Pruning	71.945	2.727	82.828	84.722	34.081	23.958
	Baseline	0.542	0.141	0.137	0.181	0.123	0.201
	Global Unstructured Pruning	0.296	0.037	0.546	0.861	0.297	0.132
	KMeans-Based Pruning	0.302	0.047	0.413	0.831	0.269	0.164
Rotational RPE (°)	L1 Unstructured Pruning	0.473	0.039	0.529	0.899	0.315	0.115
	L2 Structured Pruning	0.437	0.035	0.555	0.913	0.327	0.116
	Random Structured	0.346	0.037	0.532	0.818	0.306	0.113
	Sensitivity-Based Pruning	0.346	0.057	0.563	0.907	0.324	0.120
	Baseline	0.310	0.174	0.263	0.251	0.282	0.783
	Global Unstructured Pruning	0.466	0.307	0.745	0.542	0.908	0.674
	KMeans-Based Pruning	0.382	0.378	0.732	0.615	1.076	0.735
	L1 Unstructured Pruning	0.496	0.475	0.684	0.575	0.955	0.617
	L2 Structured Pruning	0.492	0.170	0.697	0.501	0.883	0.640
	Random Structured	0.338	0.220	0.680	0.633	0.995	0.643
	Sensitivity-Based Pruning	0.423	0.234	0.647	0.533	0.933	0.609

Based on both trajectory, accuracy and practical considerations like inference time, we identify the most promising pruning strategies below:

Random Structured Pruning stands out as the most robust pruning method. It consistently maintained low translational error (e.g., 29.67% on sequence 01 vs. 23.67% for the baseline) and delivered strong results in terms of ATE (as low as 11.49m on sequence 10), RPE, and rotational accuracy. Importantly, Random Structured Pruning achieves this while maintaining inference speeds comparable to the baseline, as its structured nature reduces entire neuron paths — simplifying runtime computations on standard hardware. Additionally, Random Structured Pruning demonstrates superior generalization across diverse driving scenarios, suggesting its resilience to varying motion dynamics and scene complexities.

Sensitivity-Based Pruning also delivered competitive results, particularly in shorter or less dynamic sequences. It showed improved RPE rotational metrics in sequence 10 (0.609° vs. 0.783° baseline), indicating its ability to retain critical orientation features. Its inference time is marginally slower than Random Structured due to retaining more unstructured sparsity, which is harder to optimize during runtime.

In contrast, **Global Unstructured** and **L1 Unstructured Pruning**, though effective in compressing the model, suffered considerable accuracy degradation. For instance, on sequence 05, the translational error jumped from 9.437% (baseline) to over 50% in both cases, suggesting the loss of critical localized features during unstructured weight removal.

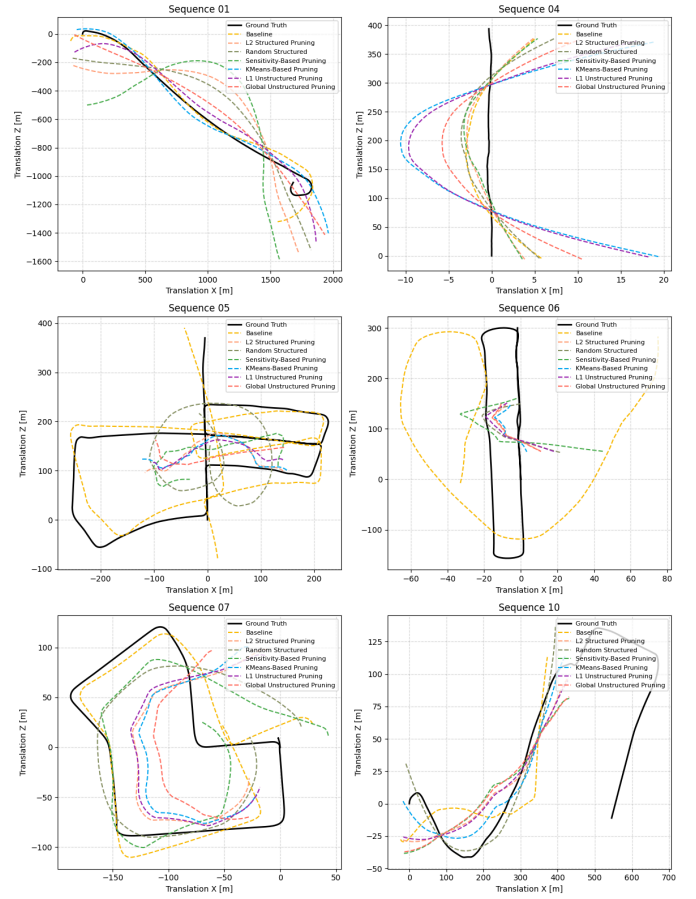


Fig. 3. Trajectories for the pruned models, TSformer-VO-2 (baseline), and ground truth on 8 sequences of the KITTI dataset

Model quantization

Static quantization introduced a consistent and significant drop in model accuracy compared to the baseline. However, performance remained largely unchanged across the three precision levels—float16, float32, and float64—as seen from the steady metrics in Table V and the overlapping trajectories in Figure 4. This indicates that vision transformers are robust to quantization within the floating-point range, and that the quantization precision itself has minimal effect on performance, at least until a more aggressive threshold is crossed.

Interestingly, even at the highest floating-point precision (float64), the quantized models exhibited a noticeable decrease in accuracy compared to the baseline. For example, the translational error in Sequence 05 rose sharply from 9.437% (baseline) to 55.799% under all static quantized models. A similar pattern was observed in ATE, RPE, and rotational error metrics across all tested sequences. This consistent degradation suggests that static quantization incurs a fixed cost to accuracy, likely due to the way PyTorch implements static post-training quantization through `torch.quantization.convert` and model-level casting using `model.to(torch.float)`. These methods may not account for the layer-wise sensitivity of attention blocks and temporal embeddings, which are crucial for accurate visual odometry.

Our static quantization pipeline relied on PyTorch’s default post-training calibration strategy, using a representative subset of the KITTI dataset. While effective for general quantization, this method may fail to capture the distributional diversity across motion patterns and scene dynamics, especially in a temporally sensitive task like visual odometry. This may

explain the fixed performance drop observed across all static precisions.

Despite these drawbacks, float16 emerges as the most practical format among the static quantization options. Since all three float variants produced nearly identical metrics across all sequences (e.g., RPE, ATE, Translational Error), adopting float16 can halve model size and reduce memory usage without sacrificing any additional accuracy compared to float32 or float64.

TABLE V
ACCURACY METRICS OF QUANTIZATION APPROACHES

Quantization Approach		Sequence					
		01	04	05	06	07	10
Translational Error (%)	Baseline	23.671	9.035	9.437	17.701	13.998	55.308
	Post-training Static - float16	30.405	4.529	55.796	61.942	58.478	55.319
	Post-training Static - float32	30.414	4.516	55.799	61.944	58.479	55.308
	Post-training Static - float64	30.414	4.516	55.799	61.944	58.479	55.308
	Dynamic Fixed-Point - qint8	46.304	15.879	51.252	63.740	67.503	68.499
Rotational error (°/100m)	Baseline	5.855	4.874	3.907	5.395	7.440	36.195
	Post-training Static - float16	8.176	4.963	43.763	33.033	46.728	36.206
	Post-training Static - float32	8.178	4.950	43.764	33.029	46.729	36.195
	Post-training Static - float64	8.178	4.950	43.764	33.029	46.729	36.195
	Dynamic Fixed-Point - qint8	13.197	20.187	36.414	45.034	76.253	42.760
ATE (m)	Baseline	101.699	6.005	37.679	46.788	23.161	30.395
	Post-training Static - float16	54.360	1.138	74.352	72.697	27.121	30.393
	Post-training Static - float32	54.350	1.135	74.323	72.688	27.122	30.395
	Post-training Static - float64	54.350	1.135	74.323	72.688	27.122	30.395
	Dynamic Fixed-Point - qint8	173.614	4.037	79.455	79.723	28.288	18.483
Translational RPE (m)	Baseline	0.542	0.141	0.137	0.181	0.123	0.201
	Post-training Static - float16	0.471	0.108	0.439	0.637	0.273	0.201
	Post-training Static - float32	0.471	0.108	0.439	0.636	0.273	0.201
	Post-training Static - float64	0.471	0.108	0.439	0.636	0.273	0.201
	Dynamic Fixed-Point - qint8	0.457	0.041	0.440	0.801	0.281	0.113
Rotational RPE (°)	Baseline	0.310	0.174	0.263	0.251	0.282	0.783
	Post-training Static - float16	0.418	0.200	1.019	0.760	1.209	0.783
	Post-training Static - float32	0.418	0.200	1.019	0.760	1.209	0.783
	Post-training Static - float64	0.418	0.200	1.019	0.760	1.209	0.783
	Dynamic Fixed-Point - qint8	0.484	0.323	0.714	0.655	1.037	0.651

Dynamic quantization, implemented via PyTorch’s `torch.ao.quantization.quantize_dynamic`, was tested using the qint8 precision. This form of quantization, supported via QNNPACK on MacOS as of PyTorch 2025, resulted in substantial accuracy degradation, particularly on longer or more dynamic sequences. For instance, translational error increased from 23.67% to 46.30% in Sequence 01 and from 17.70% to 63.74% in Sequence 06. ATE and RPE metrics also suffered: ATE in Sequence 01 rose from 101.7 m (baseline) to 173.6 m (qint8), and translational RPE in Sequence 06 nearly quadrupled. These results highlight the limitations of aggressive quantization when applied without quantization-aware training (QAT) or layer-wise sensitivity control.

One noteworthy anomaly appeared in Sequence 04, where all static quantized models actually outperformed the baseline. ATE dropped from 6.005m (baseline) to 1.135m, and translational error was halved. This could be attributed to the sequence’s lower complexity or coincidental alignment with calibration data. While intriguing, further testing is required to determine whether this effect is replicable or data-specific.

From a performance standpoint, Table III indicates that statically quantized models achieved 10–29% faster inference relative to the baseline. Model sizes also scaled linearly with datatype precision, as expected. The doubling of model size from float16 to float32 and float64 reflects the fact that quantization primarily affects weight storage, while activations and intermediate computations remain at their original precision during inference.

Taken together, these results demonstrate that:

- 1) Static float16 quantization offers the best trade-off between memory efficiency and accuracy.

- 2) Float precision (16/32/64) does not significantly affect performance, validating float16 as a lightweight deployment option.
- 3) Dynamic qint8 quantization, while promising for extreme compression, currently lacks robustness for high-accuracy visual odometry tasks and requires further refinement or QAT.

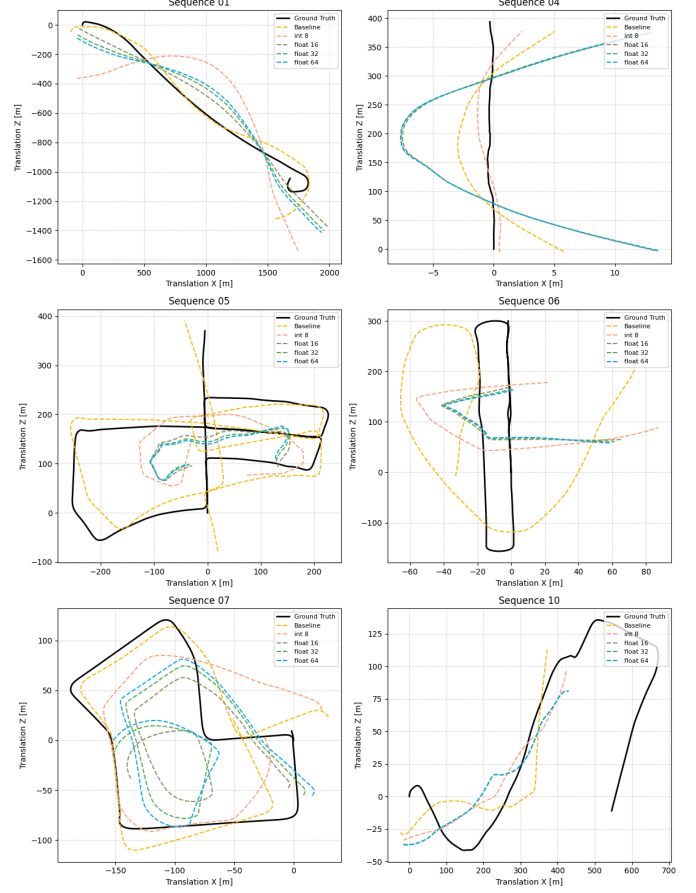


Fig. 4. Trajectories for the quantized models, TSformer-VO-2 (baseline), and ground truth on 8 sequences of the KITTI dataset

IV. DISCUSSION

This study aimed to optimize vision transformers (ViTs) for low-power visual odometry (VO) by systematically evaluating post-training pruning and quantization techniques. Our objective was to identify practical strategies for reducing computational and memory demands while preserving trajectory estimation accuracy—a critical requirement for deploying VO on resource-constrained devices. Below, we contextualize our findings within existing literature, address their implications, and outline limitations.

Quantization Robustness

The resilience of vision transformers to high-precision quantization (float64 to float16) aligns with prior studies on transformer architectures in other domains. For instance, Rokh et al. (2023) demonstrated that transformers exhibit tolerance to reduced precision due to their self-attention mechanisms, which prioritize relative relationships over absolute numerical precision. However, the abrupt accuracy collapse at qint8 (dynamic quantization) mirrors observations in CNN-based visual odometry systems, where extreme quantization disrupts gradient flow and feature map continuity (Zhan et al., 2019). The fixed accuracy penalty observed in static quantization (even at float64) may stem from PyTorch’s implementation,

which enforces uniform scaling factors across layers, potentially misaligning with the heterogeneous sensitivity of spatial-temporal attention layers in TSformer-VO.

Pruning Sensitivity

The accuracy degradation from pruning 20–30% of weights underscores the dense interdependence of parameters in vision transformers. This contrasts with CNNs, where magnitude-based pruning can remove >50% of weights without significant loss (Vadera & Ameen, 2022). Vision transformers’ reliance on global attention mechanisms likely amplifies the impact of pruning: removing “low-magnitude” weights may disrupt long-range dependencies critical for pose regression. This aligns with findings in NLP transformers, where unstructured pruning disproportionately harms performance compared to structured head pruning (Michel et al., 2019).

Efficiency-Accuracy Trade-offs

Our results advocate prioritizing quantization over pruning for initial optimization—a strategy supported by the minimal accuracy loss at float16 (model size reduced by 50%) versus the steep penalties of moderate pruning. This mirrors trends in CNNs but contrasts with recurrent architectures (e.g., LSTMs), where pruning is often more effective (Rokh et al., 2023). The inference speed gains (10–29%) from quantization further validate this approach, though real-world deployment may yield variable results depending on hardware support for low-precision arithmetic.

Limitations

Our study has several limitations. First, experiments were restricted to the KITTI dataset, which primarily represents outdoor driving scenarios. Indoor environments (e.g., with rapid rotations or occlusions) or dynamic scenes (e.g., drones navigating cluttered spaces) may exacerbate accuracy losses from quantization and pruning, as shown in recent work on VO robustness (Zhang et al., 2024). Second, implementation artifacts, such as PyTorch’s static quantization tools, may introduce suboptimal scaling factors for attention layers, inflating the observed fixed accuracy penalty. Third, our post-training optimization approach precludes recovery of lost accuracy through fine-tuning, unlike methods such as quantization-aware training (QAT), which adapts models to low precision during training. Finally, hardware-specific factors—such as the AMD Ryzen 5 7530U’s x86 architecture versus ARM-based edge devices—limit the generalizability of our inference speed results, as memory access patterns and instruction sets differ significantly across platforms.

Practical recommendations

Based on our findings, developers should adopt a tiered optimization strategy. First, prioritize quantization to float16, leveraging hardware acceleration where available, as this reduces model size by 50% with minimal accuracy loss. Second, apply pruning sparingly (<20%), focusing on structured removal of redundant attention heads rather than unstructured weight pruning, to preserve spatial-temporal relationships. Third, profile layer-wise sensitivity to quantization, as spatial and temporal attention layers may require heterogeneous precision—for example, using higher precision for positional embeddings critical to trajectory coherence. Finally, validate optimized models on target hardware early in the deployment pipeline, as runtime performance gains depend heavily on platform-specific

support for low-precision operations and pruning-induced sparsity.

V. CONCLUSIONS

This work demonstrates that vision transformers for visual odometry can be optimized for low-power deployment via quantization and pruning, albeit with distinct trade-offs. Quantization to float16 reduces model size by half with minimal accuracy loss, while pruning beyond 20% of weights degrades trajectory estimation significantly. These findings align with broader trends in transformer optimization but highlight unique challenges in visual odometry, where pose regression demands precise spatial-temporal feature preservation.

Our results suggest that developers can achieve viable efficiency gains by prioritizing quantization, which incurs a predictable accuracy penalty, before cautiously applying pruning. However, the fixed accuracy cost of static quantization warrants further investigation into layer-wise adaptive methods.

Future work could focus on more advanced pruning and quantization methods, as well as hybrid approaches that combine the 2 strategies. Lightweight fine-tuning could also be explored for recovering the lost accuracy from quantization, but this may add slight overhead to the model development process.

By bridging the gap between transformer capabilities and low-power constraints, this work advances the deployment of advanced odometry systems in resource-constrained robotics, enabling broader adoption in applications like micro-drones and wearable navigation aids.

REFERENCES

- [1] Z. Teed et al., “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” *arXiv preprint arXiv:2108.10869*, 2021.
- [2] W. Wang et al., “TartanVO: A Generalizable Learning-based VO,” *arXiv preprint arXiv:2011.00359*, 2021.
- [3] C. Campos et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *arXiv preprint arXiv:2007.11898*, 2020.
- [4] V. Tankovich et al., “HITNet: Hierarchical Iterative Tile Refinement Network for Real-time Stereo Matching,” *arXiv preprint arXiv:2007.12140*, 2020.
- [5] L. Li et al., “Deep Learning Based Method for Pruning Deep Neural Networks,” in *Proc. IEEE Int. Conf. Multimedia & Expo Workshops (ICMEW)*, 2019, pp. 68–71, doi: 10.1109/ICMEW.2019.00-68.
- [6] T. Liang et al., “Pruning and Quantization for Deep Neural Network Acceleration: A Survey,” *arXiv preprint arXiv:2101.09671*, 2021.
- [7] A. O. Françani and M. R. O. A. Maximo, “Transformer-Based Model for Monocular Visual Odometry,” *IEEE Access*, vol. 13, pp. 11045–11056, 2025.
- [8] G. Bertasius et al., “Is Space-Time Attention All You Need for Video Understanding?” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021.
- [9] S. Vadera and S. Ameen, “Methods for Pruning Deep Neural Networks,” *IEEE Access*, vol. 10, pp. 63280–63300, 2022, doi: 10.1109/ACCESS.2022.3182659.

- [10] B. Rokh *et al.*, “A Comprehensive Survey on Model Quantization for Deep Neural Networks in Image Classification,” *ACM Trans. Intell. Syst. Technol.*, vol. 14, no. 6, pp. 1–50, 2023, doi: 10.1145/3632517.
- [11] K. Tsubota and K. Aizawa, “Comprehensive Comparisons of Uniform Quantization in Deep Image Compression,” *IEEE Access*, vol. 11, pp. 4455–4465, 2023, doi: 10.1109/ACCESS.2023.3236086.
- [12] A. Luqman *et al.*, “Post-Training Non-Uniform Quantization for Convolutional Neural Networks,” *arXiv preprint arXiv:2412.07391*, 2020.
- [13] Zhan, Huangying (2019). “KITTI Odometry Evaluation Toolbox.” Available: <https://github.com/Huangying-Zhan/kitti-odom-eval>
- [14] J. Wu *et al.*, “Integer Quantization for Deep Learning Inference,” *arXiv preprint arXiv:2004.09602*, 2020.